



Nombre:

Ambar Crystal Mercedes Ramírez

Matrícula:

2025-1373

Asignatura:

Electiva 1

Periodo:

2026-C-2

Profesor:

Francis Ramirez

Tema:

Actividad 1: Desarrollo del Proceso ETL en .NET(Arquitectura)

Creación de la Arquitectura y Desarrollo del Proceso de Extracción ETL

Sistema de Análisis de Ventas

1. Introducción

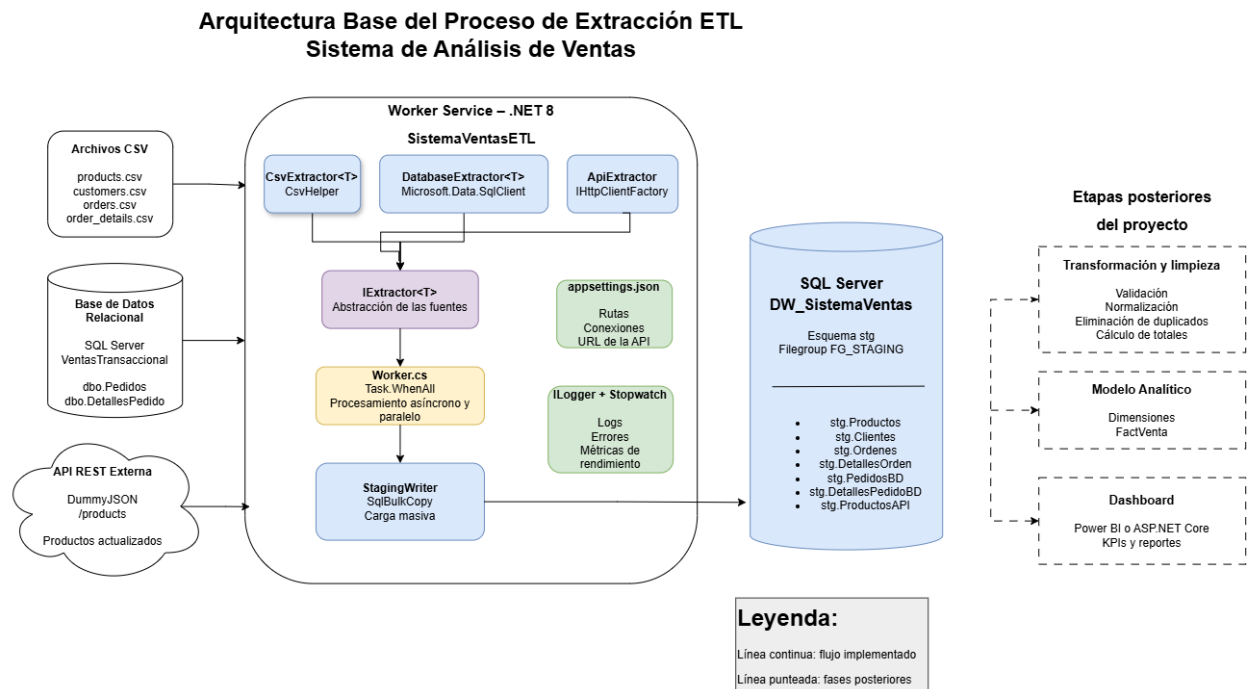
En esta práctica desarrollé la arquitectura base y el proceso de extracción del Sistema de Análisis de Ventas con Proceso ETL. La solución fue implementada mediante un Worker Service en .NET 8, encargado de obtener información desde archivos CSV, una base de datos relacional y una API REST externa.

Los datos extraídos se almacenan temporalmente en tablas del esquema staging de la base de datos analítica **DW_SistemaVentas**. Para garantizar un proceso organizado y mantenible, la solución utiliza interfaces, clases especializadas para cada fuente, inyección de dependencias, procesamiento asíncrono, carga masiva y registro de eventos mediante **ILogger**.

2. Objetivo

El objetivo de esta implementación es desarrollar el proceso de extracción de una solución ETL multifuente, permitiendo integrar datos de archivos CSV, SQL Server y una API REST en una zona staging centralizada. Además, se busca que la arquitectura cumpla con atributos de rendimiento, escalabilidad, seguridad y mantenibilidad.

3. Arquitectura



La arquitectura está compuesta por tres fuentes de datos, un Worker Service desarrollado en .NET 8 y la base de datos analítica DW_SistemaVentas.

Los archivos CSV son procesados mediante CsvExtractor<T>, la base de datos relacional VentasTransaccional es consultada mediante DatabaseExtractor<T> y la API REST externa es consumida mediante ApiExtractor.

Todos los extractores implementan la interfaz genérica IExtractor<T>, permitiendo que las fuentes puedan ser reemplazadas o ampliadas sin modificar de manera significativa el resto de la aplicación.

El archivo Worker.cs coordina las extracciones y cargas mediante Task.WhenAll. Posteriormente, StagingWriter utiliza SqlBulkCopy para insertar los datos de forma masiva en las tablas staging.

Las etapas de transformación, carga al modelo analítico y visualización se presentan con líneas punteadas debido a que corresponden a fases posteriores del proyecto.

4. Fuentes de datos

Fuente	Tecnología	Información extraída
Archivos CSV	CsvHelper	Productos, clientes, órdenes y detalles
Base de datos relacional	Microsoft.Data.SqlClient	Pedidos y detalles históricos
API REST externa	IHttpClientFactory	Productos actualizados de DummyJSON

Los archivos CSV utilizados fueron products.csv, customers.csv, orders.csv y order_details.csv.

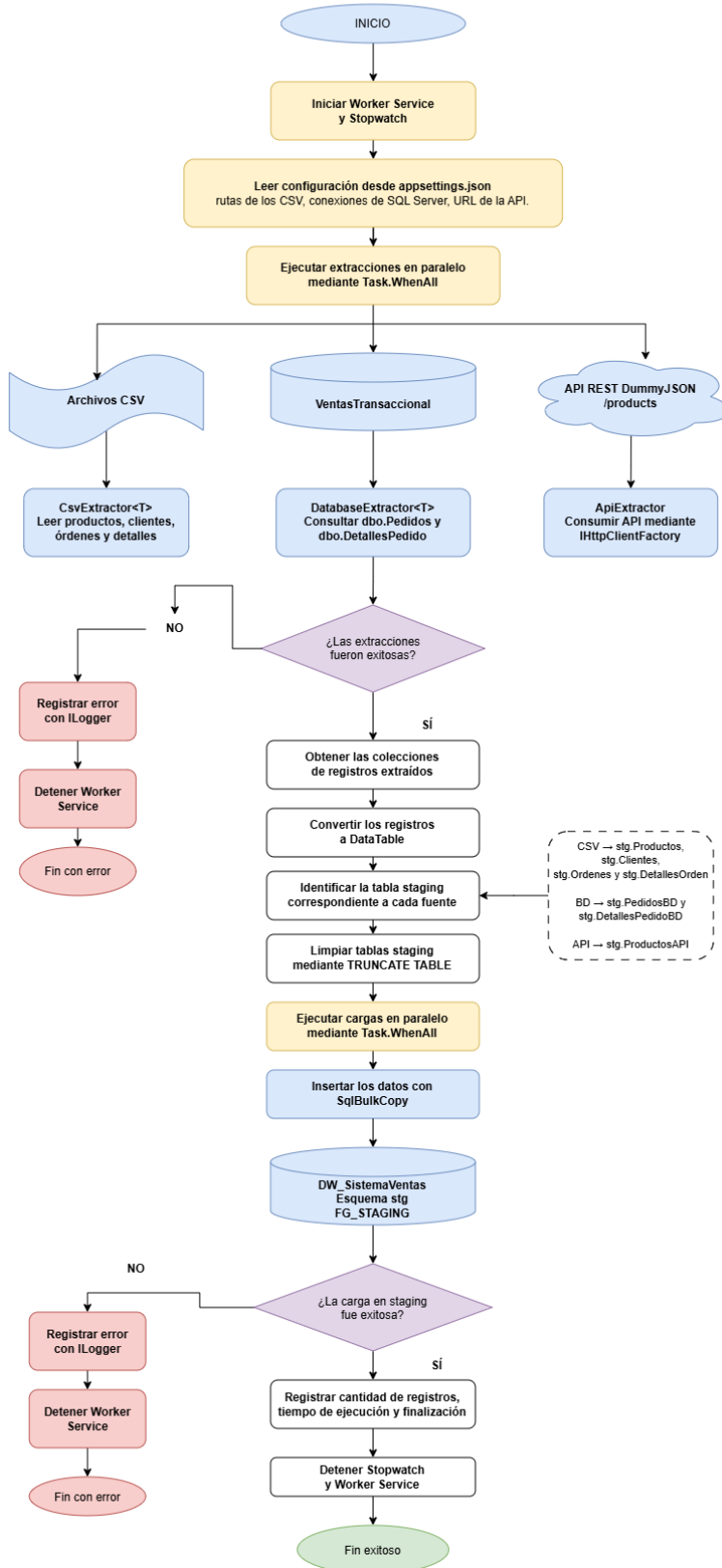
La fuente relacional utilizada fue la base de datos VentasTransaccional, específicamente las tablas dbo.Pedidos y dbo.DetallesPedido.

Para la fuente REST se utilizó la API pública DummyJSON, consultando el endpoint /products. Los productos recibidos fueron almacenados independientemente para conservar la trazabilidad de su origen.

5. Flujo del proceso de extracción

Diagrama de Flujo del Proceso de Extracción ETL

Sistema de Análisis de Ventas



El proceso comienza con la inicialización del Worker Service y del componente Stopwatch. A continuación, se leen las rutas, cadenas de conexión y URL de la API desde appsettings.json.

Las extracciones desde CSV, SQL Server y la API REST se ejecutan de manera asíncrona y paralela mediante Task.WhenAll. Si alguna extracción falla, la excepción es registrada mediante ILogger y la ejecución es detenida.

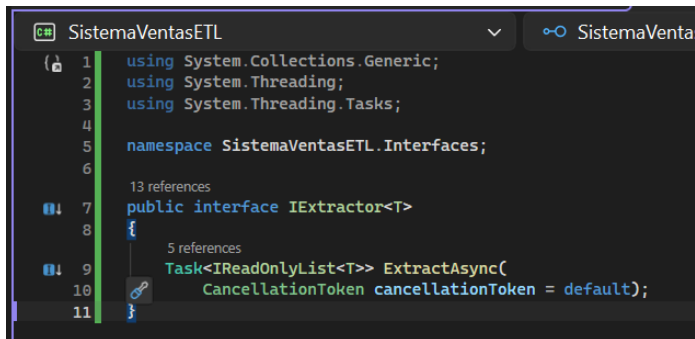
Cuando las extracciones finalizan correctamente, los registros son convertidos en objetos DataTable. El sistema identifica la tabla staging correspondiente, elimina los registros anteriores mediante TRUNCATE TABLE y ejecuta la carga masiva utilizando SqlBulkCopy.

Finalmente, se registran la cantidad de registros procesados, el tiempo de ejecución y la finalización del Worker Service.

6. Componentes implementados

6.1 IExtractor<T>

La interfaz IExtractor<T> define el contrato común para todas las fuentes de datos. Su método ExtractAsync devuelve una colección de registros de forma asíncrona y permite recibir un CancellationToken.



```
1 using System.Collections.Generic;
2 using System.Threading;
3 using System.Threading.Tasks;
4
5 namespace SistemaVentasETL.Interfaces;
6
7 public interface IExtractor<T>
8 {
9     Task<IReadOnlyList<T>> ExtractAsync(
10         CancellationToken cancellationToken = default);
11 }
```

6.2 CsvExtractor<T>

El componente CsvExtractor<T> utiliza la biblioteca CsvHelper para leer y convertir las filas de los archivos CSV en objetos de C#. También valida la existencia de cada archivo y registra el inicio, finalización, cantidad de registros y errores.

```

7 using System.Globalization;
8
9 namespace SistemaVentasETL.Extractors;
10
11 7 references
12 public class CsvExtractor<T> : IExtractor<T>
13 {
14     private readonly IConfiguration _configuration;
15     private readonly ILogger<CsvExtractor<T>> _logger;
16
17     0 references
18     public CsvExtractor(
19         IConfiguration configuration,
20         ILogger<CsvExtractor<T>> logger)
21     {
22         _configuration = configuration;
23         _logger = logger;
24     }
25
26     5 references
27     public async Task<IReadOnlyList<T>> ExtractAsync(
28         CancellationToken cancellationToken = default)
29     {
30         string filePath = GetFilePath();
31
32         if (string.IsNullOrWhiteSpace(filePath))
33         {
34             throw new InvalidOperationException(
35                 $"No se encontró una ruta configurada para {typeof(T).Name}.");
36         }
37
38         if (!File.Exists(filePath))
39         {
40             throw new FileNotFoundException(
41                 $"No se encontró el archivo CSV: {filePath}");
42         }
43
44         _logger.LogInformation(
45             "Iniciando extracción del archivo {filePath}.",
46             filePath);
47     }
48 }

```

6.3 DatabaseExtractor<T>

El componente DatabaseExtractor<T> utiliza Microsoft.Data.SqlClient para conectarse a VentasTransaccional y ejecutar consultas asíncronas sobre dbo.Pedidos y dbo.DetallesPedido.

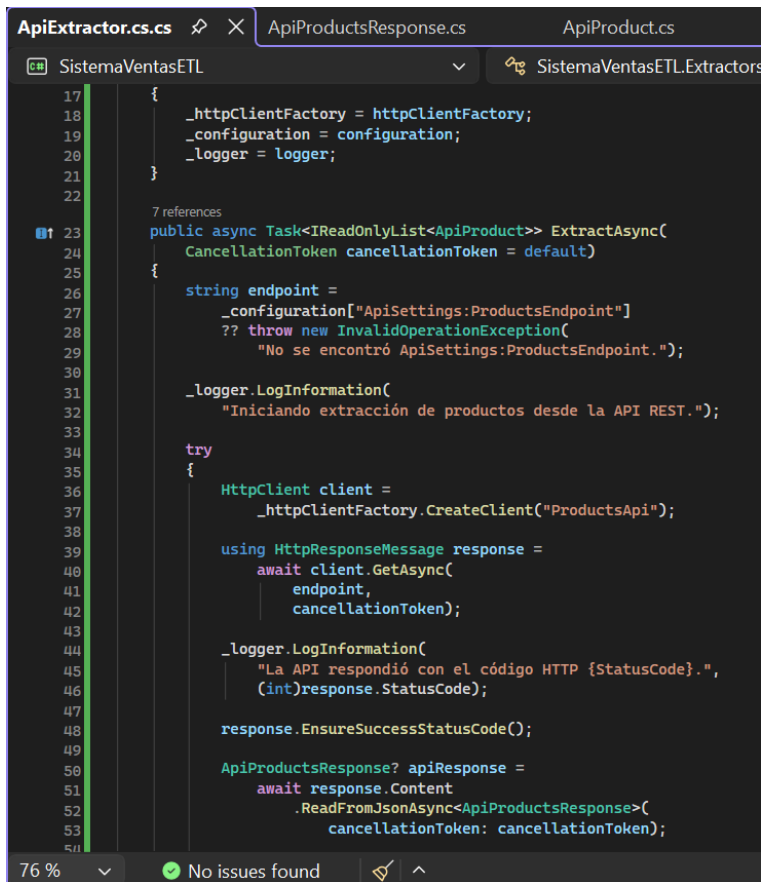
```

DatabaseExtractor.cs HistoricalOrderDetail.cs HistoricalOrder.cs StagingWriter.cs
SistemaVentasETL SistemaVentasETL.Extractors.DatabaseExtractor Extract
168
169 await using SqlDataReader reader =
170     await command.ExecuteReaderAsync(cancellationToken);
171
172 while (await reader.ReadAsync(cancellationToken))
173 {
174     details.Add(new HistoricalOrderDetail
175     {
176         OrderDetailID = reader.GetInt64(
177             reader.GetOrdinal("DetallePedidoId")),
178         OrderID = reader.GetInt32(
179             reader.GetOrdinal("PedidoId")),
180         ProductID = reader.GetInt32(
181             reader.GetOrdinal("ProductoId")),
182         Quantity = reader.GetInt32(
183             reader.GetOrdinal("Cantidad")),
184         UnitPrice = reader.GetDecimal(
185             reader.GetOrdinal("PrecioUnitario")),
186         TotalPrice = reader.GetDecimal(
187             reader.GetOrdinal("PrecioTotal")),
188         CreatedAt = reader.GetDateTime(
189             reader.GetOrdinal("FechaCreacion"))
190     });
191 }
192
193 _logger.LogInformation(
194     "Extracción completada. Se leyeron {RecordCount} detalles desde la base de datos.",
195     details.Count);
196
197 return details;
198
76 % No issues found

```

6.4 ApiExtractor

El componente ApiExtractor utiliza IHttpClientFactory para consumir la API REST. La respuesta JSON es deserializada en objetos ApiProduct, validando previamente que la respuesta HTTP sea exitosa.

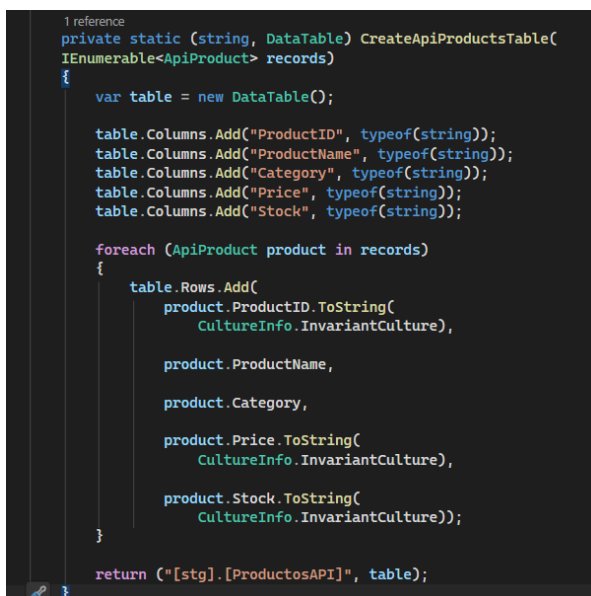


```
ApiExtractor.cs.cs  ApiProductsResponse.cs  ApiProduct.cs
SistemaVentasETL
17 {
18     _httpClientFactory = httpClientFactory;
19     _configuration = configuration;
20     _logger = logger;
21 }
22
23 7 references
24 public async Task<IReadOnlyList<ApiProduct>> ExtractAsync(
25     CancellationToken cancellationToken = default)
26 {
27     string endpoint =
28         _configuration["ApiSettings:ProductsEndpoint"]
29         ?? throw new InvalidOperationException(
30             "No se encontró ApiSettings:ProductsEndpoint.");
31
32     _logger.LogInformation(
33         "Iniciando extracción de productos desde la API REST.");
34
35     try
36     {
37         HttpClient client =
38             _httpClientFactory.CreateClient("ProductsApi");
39
40         using HttpResponseMessage response =
41             await client.GetAsync(
42                 endpoint,
43                 cancellationToken);
44
45         _logger.LogInformation(
46             "La API respondió con el código HTTP {StatusCode}.",
47             (int)response.StatusCode);
48
49         response.EnsureSuccessStatusCode();
50
51         ApiProductsResponse? apiResponse =
52             await response.Content
53                 .ReadFromJsonAsync<ApiProductsResponse>(
54                     cancellationToken);
55     }
56 }
```

76 % No issues found

6.5 StagingWriter

El servicio StagingWriter convierte los registros extraídos a objetos DataTable y utiliza SqlBulkCopy para realizar inserciones masivas. Antes de cada carga se ejecuta TRUNCATE TABLE para evitar duplicados provocados por ejecuciones anteriores.



```
1 reference
private static (string, DataTable) CreateApiProductsTable(
    IEnumerable<ApiProduct> records)
{
    var table = new DataTable();

    table.Columns.Add("ProductID", typeof(string));
    table.Columns.Add("ProductName", typeof(string));
    table.Columns.Add("Category", typeof(string));
    table.Columns.Add("Price", typeof(string));
    table.Columns.Add("Stock", typeof(string));

    foreach (ApiProduct product in records)
    {
        table.Rows.Add(
            product.ProductID.ToString(
                CultureInfo.InvariantCulture),

            product.ProductName,

            product.Category,

            product.Price.ToString(
                CultureInfo.InvariantCulture),

            product.Stock.ToString(
                CultureInfo.InvariantCulture));
    }

    return ("[stg].[ProductosAPI]", table);
}
```

6.6 Worker.cs

El Worker coordina las extracciones y cargas mediante Task.WhenAll. También utiliza Stopwatch para medir el rendimiento y ILogger para registrar eventos, cantidades procesadas y errores.

```
Microsoft Visual Studio Debu...
info: SistemaVentasETL.Services.StagingWriter[0]
      Iniciando carga de 20000 registros en [stg].[PedidosBD].
info: SistemaVentasETL.Services.StagingWriter[0]
      Carga completada. Se insertaron 2000 registros en [stg].[Productos].
info: SistemaVentasETL.Services.StagingWriter[0]
      Carga completada. Se insertaron 5000 registros en [stg].[Clientes].
info: SistemaVentasETL.Services.StagingWriter[0]
      Carga completada. Se insertaron 20000 registros en [stg].[Ordenes].
info: SistemaVentasETL.Services.StagingWriter[0]
      Carga completada. Se insertaron 20000 registros en [stg].[PedidosBD].
info: SistemaVentasETL.Services.StagingWriter[0]
      Iniciando carga de 60157 registros en [stg].[DetallesPedidoBD].
info: SistemaVentasETL.Services.StagingWriter[0]
      Iniciando carga de 194 registros en [stg].[ProductosAPI].
info: SistemaVentasETL.Services.StagingWriter[0]
      Carga completada. Se insertaron 194 registros en [stg].[ProductosAPI].
info: SistemaVentasETL.Services.StagingWriter[0]
      Carga completada. Se insertaron 60161 registros en [stg].[DetallesOrden].
info: SistemaVentasETL.Services.StagingWriter[0]
      Carga completada. Se insertaron 60157 registros en [stg].[DetallesPedidoBD].
info: SistemaVentasETL.Worker[0]
      Carga de las tres fuentes en staging completada en 1511 ms.
info: SistemaVentasETL.Worker[0]
      Proceso ETL de extracción finalizado correctamente en 2438 ms.
info: Microsoft.Hosting.Lifetime[0]
      Application is shutting down...

D:\ProyectoSistemaVentas\ETL\SistemaVentasETL\bin\Debug\net8.0\SistemaVentasETL.exe
0x0).
Press any key to close this window . . .|
```

7. Tablas staging

Fuente	Tabla staging
CSV de productos	stg.Productos
CSV de clientes	stg.Clientes
CSV de órdenes	stg.Ordenes
CSV de detalles	stg.DetallesOrden
Base de datos, pedidos	stg.PedidosBD
Base de datos, detalles	stg.DetallesPedidoBD
API REST	stg.ProductosAPI

Las tablas staging se encuentran en el esquema stg y utilizan el filegroup FG_STAGING. Las fuentes fueron almacenadas en tablas separadas para conservar la trazabilidad y evitar que los datos provenientes de una fuente reemplacen los registros de otra.


```

GO
SELECT
    TABLE_SCHEMA AS Esquema,
    TABLE_NAME AS Tabla,
    ORDINAL_POSITION AS Orden,
    COLUMN_NAME AS Columna,
    DATA_TYPE AS TipoDato,
    CHARACTER_MAXIMUM_LENGTH AS Longitud
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'stg'
AND TABLE_NAME IN ('PedidosBD', 'DetallesPedidoBD')
ORDER BY TABLE_NAME, ORDINAL_POSITION;
GO

```

Esquema	Tabla	Filegroup
1	stg	DetallesPedidoBD
2	stg	PedidosBD

Esquema	Tabla	Orden	Columna	TipoDato	Longitud
1	stg	DetallesPedidoBD	1	DetallePedidoid	nvarchar
2	stg	DetallesPedidoBD	2	Pedidoid	nvarchar
3	stg	DetallesPedidoBD	3	Productoid	nvarchar
4	stg	DetallesPedidoBD	4	Cantidad	nvarchar
5	stg	DetallesPedidoBD	5	PrecioUnitario	nvarchar
6	stg	DetallesPedidoBD	6	PrecioTotal	nvarchar
7	stg	DetallesPedidoBD	7	FechaCreacion	nvarchar
8	stg	PedidosBD	1	Pedidoid	nvarchar

Query executed successfully.

8. Resultados obtenidos

Fuente y conjunto	Registros
CSV – Productos	2,000
CSV – Clientes	5,000
CSV – Órdenes	20,000
CSV – Detalles de órdenes	60,161
Base de datos – Pedidos	20,000
Base de datos – Detalles de pedidos	60,157
API REST – Productos	194
Total	167,512

La ejecución final permitió extraer y almacenar correctamente 167,512 registros provenientes de las tres fuentes. Los conteos obtenidos en las tablas staging coinciden con las cantidades registradas por el Worker Service.

La API REST respondió correctamente y se almacenaron 194 productos en stg.ProductosAPI. También se validaron muestras de los datos recibidos para comprobar que los campos de identificación, nombre, categoría, precio y existencia fueron almacenados correctamente.

SQLQuery10.sql - I...ivio Mercedes (75))* X SQLQuery9.sql - IN...ivio Merc

```

'API - Productos',
COUNT(*)
FROM stg_ProductosAPI;
GO

SELECT TOP (10)
ProductID,

```

100 %

Results Messages

	FuenteTabla	CantidadRegistros
1	CSV - Productos	2000
2	CSV - Clientes	5000
3	CSV - Ordenes	20000
4	CSV - DetallesOrden	60161
5	BD - Pedidos	20000
6	BD - DetallesPedido	60157
7	API - Productos	194

	ProductID	ProductName	Category	Price	Stock
1	1	Essence Mascara Lash Princess	beauty	9.99	99
2	2	Eyeshadow Palette with Mirror	beauty	19.99	34
3	3	Powder Canister	beauty	14.99	89
4	4	Red Lipstick	beauty	12.99	91
5	5	Red Nail Polish	beauty	8.99	79
6	6	Calvin Klein CK One	fragran...	49.99	29
7	7	Chanel Coco Noir Eau De	fragran...	129...	58
8	8	Dior J'adore	fragran...	89.99	98
9	9	Dolce Shine Eau de	fragran...	69.99	4
10	10	Gucci Bloom Eau de	fragran...	79.99	91

Query executed successfully.

9. Cumplimiento de los atributos de calidad

9.1 Rendimiento

El rendimiento se garantiza mediante operaciones asíncronas, ejecución paralela con Task.WhenAll y cargas masivas con SqlBulkCopy. Las inserciones se realizan utilizando lotes de 5,000 registros, evitando ejecutar una instrucción individual por cada fila.

El tiempo total de ejecución es medido mediante Stopwatch, mientras que ILogger registra la duración de las etapas de extracción y carga. La prueba final procesó 167,512 registros en un tiempo considerablemente menor al límite de cinco minutos establecido para el proyecto.

9.2 Escalabilidad

La solución utiliza una interfaz genérica IExtractor<T> y clases independientes para cada fuente. Para agregar una nueva fuente se puede implementar otro extractor y registrarlo mediante inyección de dependencias, sin realizar cambios drásticos en los componentes existentes.

Las rutas, cadenas de conexión y endpoints se encuentran centralizados en appsettings.json, facilitando la incorporación o modificación de fuentes.

9.3 Seguridad

Las credenciales y parámetros de conexión no se encuentran escritos directamente dentro de las clases. Las cadenas de conexión son obtenidas desde la configuración.

Para SQL Server se utiliza autenticación integrada de Windows mediante `Trusted_Connection=True`, por lo que no se almacena un usuario o contraseña en el código. En un ambiente de producción, la configuración sensible podría trasladarse a Secret Manager o Azure Key Vault.

9.4 Mantenibilidad

La aplicación está organizada en carpetas de modelos, interfaces, extractores y servicios. Cada clase posee una responsabilidad específica, aplicando separación de responsabilidades y principios SOLID.

La inyección de dependencias permite que el Worker utilice las interfaces sin crear directamente las implementaciones. Además, los mensajes de ILogger facilitan la identificación de errores y el seguimiento de las ejecuciones.

10. Decisiones técnicas

Elegí utilizar **CsvHelper** porque me permite relacionar los encabezados de los archivos CSV directamente con las propiedades de los modelos, facilitando la lectura y organización de los datos.

Seleccioné **Microsoft.Data.SqlClient** para conectarme a SQL Server y realizar las consultas de manera asíncrona y controlada.

Utilicé **IHttpClientFactory** para administrar correctamente la instancia de HttpClient empleada en el consumo de la API REST.

Implementé **SqlBulkCopy** porque ofrece un mejor rendimiento al cargar grandes volúmenes de registros, en comparación con la ejecución de instrucciones INSERT de forma individual.

Decidí cargar los datos de cada fuente en tablas staging independientes para conservar su origen, facilitar las validaciones posteriores y permitir una mejor identificación de posibles errores.

11. Conclusión

Mediante esta práctica desarrollé la arquitectura base y el proceso de extracción del **Sistema de Análisis de Ventas**. Implementé un Worker Service capaz de integrar correctamente datos provenientes de archivos CSV, una base de datos relacional y una API REST externa.

La solución procesa las fuentes de manera asíncrona, registra los eventos y errores, mide el tiempo de ejecución y almacena los datos en tablas staging mediante carga masiva. Además, la separación de responsabilidades y el uso de interfaces me permitieron mantener una arquitectura organizada, escalable y preparada para las próximas etapas de transformación, carga y visualización.